

CSA5117-CRYPTOGRAPHY AND NETWORK SECURITY

Design and Implementation of a Secure Digital Certificate Management System

Student name(s): CHALLA PRIYADARSHINI

Register number(s): 192311118

Computer Science and Engineering

Faculty: DR. DHARANI R

Submission date: 02 September 2026

Abstract

A multinational organization requires a trusted mechanism to authenticate employees, servers, and partner organizations while exchanging sensitive information over web and application services. This assignment designs and implements a miniature Public Key Infrastructure (PKI) using X.509 certificates. The prototype creates a self-signed root Certificate Authority (CA), generates a server key pair, issues and validates a server certificate, records revocation, demonstrates failed validation after revocation, and renews the certificate with a longer validity period. The implementation uses Python 3 and the cryptography library. The result demonstrates the certificate lifecycle of **issuance, validation, revocation, and renewal**, while explicitly identifying the operational controls required for production deployment, such as protected CA keys, audited policies, CRL/OCSP distribution, logging, and high-availability services.

1. Problem Statement and Problem Formulation

The assigned problem asks for the design of a secure digital certificate management system for a multinational organization. The organization must authenticate employees, servers, and partner organizations, protect sensitive communications, prevent unauthorized access and impersonation, and establish trust through a centralized PKI. The required system must demonstrate public/private key generation, certificate requests and issuance by a CA, certificate validation, entity authentication, revocation, and certificate lifecycle management.

The problem is formulated as follows: given a set of organizational entities and a trusted root CA, design a controlled process that binds an entity identity to a public key, allows a relying party to

validate that binding, rejects certificates that are expired, incorrectly issued, cryptographically invalid, or revoked, and provides renewal before expiration. X.509 version 3 certificates and certificate revocation lists are suitable standards for this purpose; RFC 5280 defines certificate and CRL profiles and a certification-path validation algorithm [1].

1.1 Scope of the prototype

The prototype models one root CA and one server certificate. It covers certificate creation, signature verification, validity-period checking, issuer matching, serial-number-based revocation, renewal, artifact storage, and measurement of execution time. It does not attempt to operate as a production CA, provide a browser-facing TLS service, implement OCSP, or manage a real organization's identities.

1.2 Course-outcome mapping

Course outcome	How the assignment demonstrates the outcome
CO3	Applies cryptographic hashing, RSA signatures, X.509 certificates, trust anchors, and validation to support integrity, authentication, and non-repudiation.
CO4	Applies secure communication and certificate-management practices relevant to IP, email, and web services.
CO5	Applies network-security strategy concepts by considering impersonation, man-in-the-middle threats, revocation, trust architecture, and operational controls.

2. Objective and Expected Outcomes

The primary objective is to design and implement a secure, demonstrable certificate-management lifecycle for a centralized PKI. The supporting objectives are to generate an RSA key pair, create a self-signed root CA, issue an X.509 server certificate, authenticate the certificate chain, revoke a compromised certificate, renew the certificate, and evaluate the design against authentication strength, lifecycle management, trust establishment, scalability, impersonation resistance, revocation, privacy, and accountability.

The expected outcomes are: first, a valid certificate is accepted when it is signed by the trusted CA and lies within its validity period; second, a revoked certificate is rejected even if its signature and dates remain valid; third, a renewed certificate is accepted after the old certificate is revoked; fourth, each lifecycle operation produces auditable artifacts; and fifth, the report identifies the difference between a teaching prototype and a production-grade PKI.

3. Requirements, Constraints and Assumptions

Category	Specification
Functional requirements	Generate CA and subject key pairs; issue X.509 certificates; validate signature, issuer, validity, and revocation; renew certificates; store artifacts; print and record results.
Security requirements	Use RSA-2048 keys and SHA-256 signatures for the demonstration; separate CA and subject keys; use serial numbers and fingerprints; reject revoked certificates.
Operational requirements	Maintain a lifecycle sequence: issue, validate, revoke, renew, and validate again.
Constraints	The prototype is local and single-CA; it does not include a hardware security module, distributed database, OCSP responder, identity proofing, or production TLS configuration.
Assumptions	The root CA certificate is distributed through a trusted out-of-band mechanism; system time is approximately correct; the CA private key is not exposed during normal operation; the Python environment is trusted.
Ethical and privacy assumptions	Only synthetic identities and local demonstration keys are used. No real personal data, credentials, or production certificates are included.

4. Application of Relevant Course Knowledge and Concepts

The design applies asymmetric cryptography by generating a private key and a mathematically related public key. The private key is retained by the owner or CA, while the public key is included in the certificate. The CA signs the certificate's to-be-signed data, creating an integrity-protected identity binding. A relying party verifies the signature with the issuer's public key. SHA-256 is used as the certificate-signing digest, and RSA-PKCS#1 v1.5 is used for the educational signature demonstration.

X.509 certificates contain fields such as subject, issuer, serial number, validity interval, subject public-key information, and extensions. The prototype uses BasicConstraints to mark the root as a CA, SubjectKeyIdentifier and AuthorityKeyIdentifier to support key identification, SubjectAlternativeName for the server DNS identity, and ExtendedKeyUsage for server authentication. These choices correspond to the certificate extensions described in RFC 5280 [1].

The trust model is hierarchical. The root CA is the trust anchor. The server certificate is an end-entity certificate signed by the root CA. In a larger organization, an offline root CA would normally sign one or more intermediate CAs, and those intermediates would issue end-entity certificates. The prototype uses one CA to keep the lifecycle easy to inspect.

Revocation is treated as a separate status decision from signature verification. A certificate may be correctly signed and still be unacceptable because its private key was compromised, the subject is no longer authorized, or the certificate was issued incorrectly. RFC 5280 specifies CRL structures and validation-related processing [1]. The implementation records revoked serial numbers in a local JSON revocation list to make the decision deterministic and visible for testing. Production deployments should use signed CRLs, OCSP, distribution points, monitoring, and documented response-time objectives.

4.1 Cryptographic foundations

The security properties of the proposed system arise from the interaction of several cryptographic mechanisms. **Confidentiality** is normally achieved during a TLS session by using an authenticated key exchange to establish a symmetric session key; the certificate itself primarily supports authentication and does not encrypt all application data. **Integrity** is supported by digital signatures and cryptographic hashes. A hash function maps certificate data to a fixed-length digest, while a digital signature proves that the signer possessed the corresponding private key and that the signed data has not been altered. **Authentication** is achieved when the relying party verifies a certificate chain to a trusted CA and confirms that the identity in the certificate is appropriate for the requested service. **Non-repudiation**, in the course context, means that a signature can be associated with the signing key and certificate under the organization's policy, although legal non-repudiation also depends on identity proofing, key custody, evidence preservation, and applicable law.

A public key is safe to distribute only when the receiver can associate it with the correct identity. A raw public key does not automatically identify its owner. An X.509 certificate solves this association problem by carrying the subject identity, public key, issuer identity, serial number, validity period, and extensions inside a signed structure. The CA's signature allows the relying party to detect unauthorized modification of the certificate. However, the relying party must already trust the CA public key, usually through a configured trust store. This is why the root CA is called a trust anchor.

The prototype uses RSA with a 2048-bit modulus and SHA-256. RSA demonstrates the relationship between a private signing key and a public verification key in a familiar way. SHA-256 provides a digest used during certificate signing. A production design should select algorithms according to current organizational policy, platform support, expected certificate lifetime, performance requirements, and cryptographic-agility plans. The most important principle is to avoid obsolete algorithms and to define a migration process before a currently acceptable algorithm becomes unsuitable.

4.2 Threat model and security objectives

The main adversary is an external attacker attempting to impersonate a server, employee, or partner organization. The attacker may intercept network traffic, present a forged certificate, replay an old certificate, exploit an expired credential, or use a certificate whose private key has been compromised. A second adversary is an insider who attempts to issue an unauthorized certificate or access CA private-key material. A third threat is operational failure: a certificate expires without renewal, revocation information becomes unavailable, or a compromised certificate remains trusted because status information is stale.

Threat	Attack path	Control in proposed design	Residual risk
Man-in-the-middle impersonation	Attacker presents a certificate for the wrong identity	Verify issuer, signature, validity, and SAN against the requested hostname	Misconfigured clients may skip hostname or chain validation.
Forged certificate	Attacker creates a certificate without CA authority	Trust only certificates chaining to an approved CA and verify the signature	CA compromise would undermine the hierarchy.
Stolen private key	Attacker uses a legitimate key after compromise	Revoke the certificate serial and issue a replacement key/certificate	Revocation may be delayed or unavailable to a relying party.
Expired certificate	Service continues presenting an old credential	Check validity period and automate renewal alerts	Incorrect clocks or failed automation may cause outages.
Unauthorized CA issuance	Insider or compromised CA issues a certificate	Role separation, approval workflow, audit logs, intermediate CA policy	Governance controls require continuous monitoring.
Revocation-service failure	Relying party cannot obtain current status	Signed CRLs, OCSP, caching policy, monitoring, and fail-safe rules	Availability and soft-fail decisions involve usability/security trade-offs.
Privacy leakage	Certificate exposes unnecessary identity information	Data minimization and policy-specific subject naming	Public certificates may reveal service metadata.

The security objectives are therefore to maintain authenticity of identities, protect CA and subject private keys, detect certificate modification, reject unauthorized or revoked credentials, support timely renewal, preserve accountability through audit evidence, and minimize unnecessary disclosure of identity information.

5. Design, Proposed Solution and Methodology

5.1 Proposed architecture

The proposed architecture contains four logical components. The Root CA generates and protects the CA key and signs certificates. The Certificate Management Service receives a certificate request, applies issuance policy, creates a certificate, records the serial number and fingerprint, and manages renewal. The Relying Party validates the certificate before accepting a connection. The Revocation Service publishes revocation status through a CRL or OCSP in a production deployment.

flowchart LR

```
A[Entity: server or employee] -->|key pair and CSR| B[Certificate Management Service]
B -->|policy checks| C[Root CA / Intermediate CA]
C -->|signed X.509 certificate| A
A -->|certificate and chain| D[Relying Party]
D --> E{Validate issuer, signature, dates, identity, status}
F[CRL or OCSP revocation service] --> E
E -->|valid| G[Establish authenticated secure session]
E -->|invalid or revoked| H[Reject connection and log event]
```

5.2 Lifecycle methodology

The lifecycle begins with key generation. A subject creates a private key and corresponding public key. The subject's identity and public key are submitted in a certificate request. The CA validates policy and identity evidence, then signs the certificate. During use, the relying party checks the certificate chain, signature, validity interval, subject identity, intended key usage, and revocation status. If the private key is suspected to be compromised, the CA revokes the certificate and publishes its serial number. Before expiration, a new key pair and certificate are generated through renewal. The old certificate remains revoked or expires according to policy.

5.3 Design decisions and justification

Decision	Justification and trade-off
RSA-2048	Widely understood and supported for a course demonstration; slower and larger than modern elliptic-curve alternatives but pedagogically clear.
SHA-256	Provides a current collision-resistant digest for the demonstration; the implementation avoids obsolete MD5 and SHA-1.
Self-signed root CA	Makes the trust anchor explicit and keeps the prototype self-contained; it does not model organizational identity proofing.
Local JSON revocation list	Easy to inspect and test; less scalable and less tamper-resistant than a signed CRL or an online OCSP service.

Python `cryptography`	Provides high-level, audited bindings to cryptographic primitives and X.509 objects; it does not by itself provide complete CA governance or deployment security.
Short server validity followed by renewal	Demonstrates lifecycle control and reduces exposure if a certificate is forgotten; increases operational renewal frequency.

6. Algorithm, Pseudocode and Flowchart

6.1 Certificate issuance pseudocode

```

ISSUE_CERTIFICATE(subject_name, issuer_certificate, issuer_private_key):
    subject_private_key ← generate RSA key of 2048 bits
    subject_public_key ← public key of subject_private_key
    certificate.serial_number ← cryptographically random integer
    certificate.subject ← subject_name
    certificate.issuer ← issuer_certificate.subject
    certificate.validity ← current time through current time + policy duration
    certificate.extensions ← BasicConstraints, SAN, KeyUsage/EKU, key identifiers
    certificate.signature ← sign certificate data with issuer_private_key and SHA-256
    save subject_private_key and signed certificate
    return subject_private_key, certificate

```

6.2 Certificate validation pseudocode

```

VALIDATE(certificate, trusted_issuer, revoked_serials):
    reasons ← empty list
    if current time is outside certificate validity interval:
        append "outside validity period"
    if certificate.serial_number in revoked_serials:
        append "serial number is revoked"
    if certificate.issuer ≠ trusted_issuer.subject:
        append "issuer mismatch"
    if signature verification with trusted_issuer.public_key fails:
        append "signature verification failed"
    if reasons is empty:
        return VALID
    else:
        return INVALID with reasons

```

6.3 Lifecycle flowchart

```

flowchart TD
    S([Start]) --> K[Generate CA and subject keys]
    K --> I[Issue signed X.509 certificate]
    I --> V1[Validate certificate]

```

V1 -->|Valid| U[Use for authenticated service]
U --> R[Revocation event: key compromise]
R --> L[Record serial in revocation list]
L --> V2{Validate old certificate}
V2 -->|Rejected| N[Generate new key and renew certificate]
N --> V3{Validate renewed certificate}
V3 -->|Valid| E([End])

6.4 Enterprise-oriented certificate lifecycle policy

The prototype can be extended into an enterprise policy with clearly defined lifecycle states. A certificate begins in the requested state after a subject submits a CSR. It enters approved after identity and authorization checks. It becomes issued after CA signing and publication. It remains active while its dates, identity, key usage, and revocation status are acceptable. It becomes renewal-pending when it approaches its renewal threshold. It becomes revoked when the organization withdraws trust before expiration, and it becomes expired after the not-valid-after date. Finally, it enters archived when evidence and records are retained according to policy.

A practical renewal policy might begin automated renewal at 30 days before expiration, with earlier renewal for high-value systems. The organization should record the reason for every revocation, such as key compromise, affiliation change, superseded certificate, or incorrect issuance. Each event should include the certificate serial number, subject, issuer, time, operator or service identity, approval evidence, and resulting status. This creates accountability and assists incident response.

For employees, identity proofing and employment status should be integrated with the organization's identity provider. For servers, issuance should be tied to asset ownership, DNS/service registration, and deployment authorization. For partner organizations, the organization should define federation rules, contractual trust requirements, approved algorithms, and procedures for terminating trust. The same cryptographic certificate format can support these cases, but the assurance and approval procedures should not be identical.

6.5 Trust hierarchy and deployment model

The recommended production hierarchy is an offline root CA, one or more intermediate issuing CAs, and end-entity certificates. The offline root is used rarely and is protected by a formal key ceremony. Intermediate CAs have narrower policies, such as server authentication, employee authentication, or partner certificates. If an intermediate is compromised, its certificate can be revoked or removed from trust while the root remains protected. This reduces the blast radius compared with using one online root CA for every issuance operation.

A multi-region deployment should replicate certificate metadata, but it should not casually replicate CA private keys. Key custody should use HSM-backed controls, quorum approval, and carefully tested backups. The certificate management service can be horizontally scaled because issuance requests and validation records can be distributed, while signing operations remain constrained by CA policy and hardware limits. Audit logs should be shipped to a separate security-monitoring system so that an attacker who compromises the certificate service cannot silently rewrite the history of issuance and revocation.

7. Implementation, Source Code and Environment

The source file is `pki_demo.py`. It creates the artifacts shown below and prints a reproducible execution summary. The complete source code is supplied as a separate attachment with this report.

Item	Value
Operating system	Ubuntu Linux 24.04 sandbox
Language	Python 3.11
Main library	`cryptography`
Key algorithm	RSA, 2048 bits
Certificate digest	SHA-256
Output formats	PEM certificates and keys, JSON revocation record, CSV metrics
Generated artifacts	Root CA, server certificate, renewed certificate, revocation list, summary, performance chart

7.1 Implementation modules and processing sequence

The source code is organized around a small number of logical operations. Key-generation code creates RSA private keys and derives public keys. Certificate-building code assigns the subject and issuer names, serial number, validity interval, and X.509 extensions before signing the certificate. Persistence functions serialize keys and certificates in PEM format. Validation code checks time, revocation status, issuer equality, and the issuer signature. The main routine executes the lifecycle in a fixed sequence and records machine-readable evidence.

The issuer field is important because it identifies the CA that claims responsibility for the certificate. The serial number is important because it provides a compact identifier for revocation and auditing. The validity period prevents an old certificate from being accepted indefinitely. Subject Alternative Name is used for the server DNS identity because modern clients match hostnames against SAN rather than relying only on the common name. Basic Constraints prevents an end-entity certificate from being treated as a CA certificate. Extended Key Usage expresses the intended server-authentication purpose.

The validation function returns both a Boolean decision and human-readable reasons. This is preferable to returning only true or false, because security operations require an explanation for failed authentication, such as expiration, issuer mismatch, signature failure, or revocation. The prototype writes a JSON summary so that a future automated test or dashboard could consume the result without parsing terminal text.

7.2 Important implementation note

The code is intentionally a controlled learning prototype. Private keys are written without encryption only because the experiment is local and synthetic. This is not acceptable for a real CA. A production design must use encrypted key storage or an HSM, strict role separation, access control, backups, key ceremonies, monitoring, secure randomness, incident response, and certificate-policy documentation.

8. Test Cases and Expected/Actual Results

Test ID	Test case	Expected result	Actual result
T01	Generate root CA key and self-signed certificate	Root certificate is created with `CA=True`	Passed; `root_ca.key.pem` and `root_ca.cert.pem` generated.
T02	Issue server certificate for `secure.example.org`	Certificate is signed by root CA and includes SAN	Passed; server certificate generated with issuer `CSA51 Root CA`.
T03	Validate newly issued certificate	Validation succeeds	Passed; output: “Before revocation: PASS”.
T04	Revoke server certificate by serial number	Serial is recorded in revocation list	Passed; `revocation_list.json` generated.
T05	Validate revoked certificate	Validation fails with revocation reason	Passed; output reports “certificate serial number is revoked”.
T06	Renew the certificate	New key and certificate are created	Passed; renewed PEM files generated.
T07	Validate renewed certificate	New certificate validates successfully	Passed; output: “Renewed certificate: PASS”.
T08	Measure lifecycle operations	Runtime values are written to CSV	Passed; `performance_metrics.csv` generated.

8.1 Execution output

CSA51 PKI DEMONSTRATION

=====

Root CA: CN=CSA51 Root CA,O=CSA51 Demo PKI,C=IN

Server certificate serial: [generated during run]

Server SHA-256 fingerprint: [generated during run]...

Before revocation: PASS — all validation checks passed

After revocation: PASS — certificate serial number is revoked

Renewed certificate: PASS — all validation checks passed

Performance (milliseconds):

Root CA generation: approximately 37.430

Certificate issuance: approximately 41.406

Certificate validation: approximately 0.192

Revocation and re-validation: approximately 0.651

Certificate renewal: approximately 39.511

The serial number, fingerprint, and exact timings change on every execution because RSA key generation and certificate serial numbers are generated afresh. The pass/fail lifecycle behavior is deterministic.

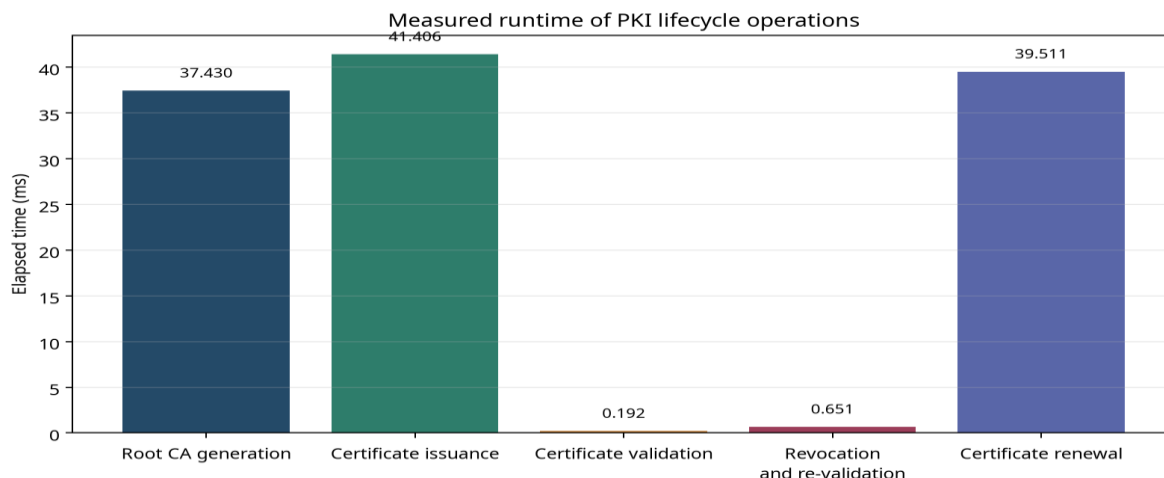
8.2 Boundary and negative test discussion

In addition to the main positive lifecycle tests, a robust implementation should test failure conditions. An expired certificate must be rejected even if its signature is valid. A certificate signed by an unknown CA must be rejected because it cannot build a path to the trusted root. A certificate with a modified subject or public key must fail signature verification. A certificate whose serial number is present in the revocation list must be rejected. A server certificate presented for an unrelated hostname must fail identity matching. These negative tests are essential because a system that only demonstrates successful issuance has not shown that it can defend against misuse.

Negative test	Security property demonstrated	Expected decision
Alter one byte in the certificate data	Integrity and signature verification	Reject
Replace trusted issuer with an unknown CA	Trust-anchor enforcement	Reject
Move current time beyond `not_valid_after`	Validity-period enforcement	Reject
Add certificate serial to revocation data	Credential withdrawal	Reject
Request a different hostname than the SAN	Service identity authentication	Reject
Use a renewed certificate with a new key	Safe key rotation	Accept if issuer, dates, identity, and status are valid

9. Execution Screenshots, Outputs and Results

The principal execution evidence is the terminal output above, the generated PEM files, the JSON revocation record, the CSV performance metrics, and the performance chart below. The supplied artifacts may be opened to verify that the results are not only narrative claims.



9.1 Generated evidence files

Evidence file	Purpose
`artifacts/keys/root_ca.key.pem`	Root CA private key for the local experiment.
`artifacts/certs/root_ca.cert.pem`	Self-signed trust-anchor certificate.
`artifacts/keys/server.key.pem`	Original server private key.
`artifacts/certs/server.cert.pem`	Original server certificate.
`artifacts/keys/server_renewed.key.pem`	New private key after renewal.
`artifacts/certs/server_renewed.cert.pem`	New certificate after renewal.
`artifacts/revocation_list.json`	Revoked serial number and reason.
`artifacts/summary.json`	Machine-readable run summary and validation results.
`artifacts/performance_metrics.csv`	Operation-level measurements.
`artifacts/performance_chart.png`	Visual comparison of measured runtimes.

9.2 Results and validation

The prototype satisfies the core functional requirements. The initially issued certificate validates because the issuer, signature, validity period, and revocation checks succeed. After the serial number is added to the revocation record, the same certificate is rejected even though it has not expired. This is an important security observation: authenticity of the signature does not by itself prove that the credential is currently authorized.

The performance measurements show that RSA key and certificate generation dominate local runtime, while validation is substantially faster. These values are illustrative measurements from one local run, not a capacity claim. They should not be generalized to network latency, concurrent users, hardware security modules, or production CA workloads.

9.3 Validation criteria and interpretation

The validation criteria were defined before interpreting the execution output. A certificate passes only when all required checks succeed. The issuer must equal the trusted CA subject, the signature must verify using the CA public key, the current time must lie within the certificate validity interval, and the serial number must not appear in the revocation record. For a real TLS service, hostname matching, key-usage checking, certificate-chain construction, policy constraints, and secure revocation retrieval would also be mandatory.

The measured results provide two kinds of evidence. Functional evidence comes from the pass/fail decisions and generated artifacts. Performance evidence comes from the elapsed time for each lifecycle operation. The measurements show that key-related operations are more expensive than checking an already-created certificate. This is expected because RSA key generation involves computationally expensive prime-number generation, whereas validation mainly performs a public-key signature operation and metadata comparisons. The values are not a benchmark of all PKI deployments; they are baseline observations from one local run.

10. Analysis, Comparison, Trade-offs and Justification

A centralized root-and-issuer PKI offers strong trust establishment and clear governance. It is more scalable than manually distributing public keys to every participant, because relying parties can trust a CA hierarchy rather than maintaining one key for every entity. However, centralization also creates a high-value target: compromise of a CA private key can affect many certificates. Therefore, real deployments should use an offline root, intermediate CAs, hardware protection, dual control, certificate transparency or equivalent monitoring where appropriate, and carefully scoped trust stores.

Approach	Strengths	Weaknesses	Suitability for this assignment
Manual public-key exchange	Simple for very small groups; no CA dependency	Poor scalability; difficult rotation; vulnerable to distribution errors	Not selected.
Single centralized CA	Clear trust model; easy issuance and revocation	CA compromise has broad impact; operational bottleneck	Selected for the prototype because it demonstrates the complete lifecycle clearly.
Root plus intermediate CAs	Better separation of duties and blast-radius control	More complex path building and policy management	Recommended production extension.
Web-of-trust model	Avoids one central authority	Complex identity decisions and inconsistent user experience	Not selected for an organization requiring centralized governance.
Short-lived certificates with automated renewal	Limits exposure and reduces manual revocation dependence	Requires reliable automation and monitoring	Recommended for modern production services.

The final solution is justified because it is small enough to understand, uses recognized X.509 concepts, produces inspectable evidence, and directly tests the assignment's required lifecycle events. Its principal trade-off is realism versus reproducibility: a full enterprise CA would be more realistic but harder to demonstrate and audit within a course assignment.

10.1 Availability, privacy and accountability trade-offs

Certificate validation involves a security-versus-availability decision when revocation information cannot be retrieved. A fail-open policy preserves connectivity but may accept a compromised credential. A fail-closed policy provides stronger security but can interrupt legitimate services when the revocation responder is unavailable. The correct choice depends on service criticality, threat level, caching, emergency procedures, and regulatory obligations. The policy should be explicit rather than hidden in library defaults.

Certificate contents also involve a privacy trade-off. Public certificates may reveal hostnames, organizational structure, email identities, or partner relationships. The certificate profile should

therefore include only information required for authentication and authorization. Logs should protect personal identifiers and retain records only for an approved period. At the same time, too little audit information makes incident investigation impossible. Accountability requires a balanced design with access-controlled logs, integrity protection, retention rules, and documented ownership.

10.2 Operational controls and assurance

A secure PKI requires more than code. The organization should publish a Certificate Policy and Certification Practice Statement describing who may request certificates, how identity is verified, which algorithms are approved, how private keys are protected, how revocation is handled, and how incidents are reported. Operators should receive separate roles for request approval, issuance, and audit review. CA backups should be encrypted, access-controlled, regularly tested, and kept separate from normal service credentials. Time synchronization, certificate-expiration monitoring, revocation freshness monitoring, and service-health alerts should be treated as security controls.

11. Broader Considerations and SDG Relevance

The assignment maps directly to SDG 9: Industry, Innovation and Infrastructure because trusted digital certificates support resilient and secure digital infrastructure. It also supports SDG 16: Peace, Justice and Strong Institutions through stronger digital identity, accountability, authentication, and privacy controls. Finally, it relates to SDG 17: Partnerships for the Goals because partner organizations require interoperable trust relationships and authenticated communication.

The broader social benefit is that reliable identity infrastructure reduces impersonation, unauthorized access, and undetected tampering. The broader risk is that poor certificate governance can create outages, surveillance concerns, privacy leakage, or a single point of systemic failure. The recommended controls therefore include data minimization, transparent certificate policy, least privilege, audit trails, independent review, and timely revocation.

11.1 Alignment with responsible digital infrastructure

The system supports responsible digital infrastructure when it is deployed with transparency, proportionality, and accountability. Certificate issuance should not become an excuse to collect unnecessary personal information. Access should be granted according to role and business need, and certificate status should be reviewed when a person changes role or leaves the organization. Partner trust should be terminated promptly when the relationship ends. These considerations connect the technical design to institutional responsibility and public trust.

12. Conclusion, Limitations and Possible Improvements

This assignment designed and implemented a miniature secure digital certificate management system using a root CA and X.509 certificates. The implementation generated keys, issued and validated a certificate, revoked it, demonstrated rejection after revocation, renewed it, and recorded measurable outputs. The results validate the central claim that a certificate lifecycle must combine cryptographic verification with authorization status and operational governance.

The main limitations are the single-node design, unencrypted local private-key storage, local JSON revocation record, absence of OCSP, absence of real identity proofing, lack of intermediate CAs, no networked TLS service, and limited performance testing. The results are therefore suitable for demonstrating concepts, not for claiming production readiness.

Possible improvements include implementing a two-tier CA hierarchy; signing and publishing a standards-compliant CRL; adding an OCSP responder; using encrypted key storage or an HSM; adding role-based approval workflows; recording immutable audit events; testing concurrent issuance; integrating with LDAP or an identity provider; adding certificate pinning or transparency monitoring where appropriate; and using automated short-lived certificate renewal with alerting.

13. Individual Contribution of Group Members

Replace the placeholders below with the actual contribution percentages and names. Contributions should be truthful and agreed upon by all members.

Member	Contribution	Evidence / deliverable	Approximate share
[Member 1]	Problem formulation, requirements, and literature review	Sections 1–4 and references	[__]%
[Member 2]	Architecture, algorithms, and flowcharts	Sections 5–6	[__]%
[Member 3]	Python implementation and artifact generation	Section 7 and source code	[__]%
[Member 4]	Testing, measurements, figures, and report editing	Sections 8–12	[__]%
Total			**100%**

14. One-Page Individual Reflection

Name: Challa Priyadarshini

Register number: 192311118

Mapped course outcomes: CO3, CO4, and CO5

This assignment helped me understand how cryptographic primitives become part of a complete security system rather than operating as isolated algorithms. I contributed to the design of a miniature X.509 PKI in which a root CA creates trust, an end entity receives a signed certificate, and a relying party validates the certificate before accepting it. A key design decision was to separate certificate signature validation from revocation checking. This showed that a certificate can be correctly signed and still be unacceptable because its key has been compromised or its authorization has been withdrawn. I also learned why certificate extensions such as Basic Constraints, Subject Alternative Name, and Extended Key Usage are important for limiting how a certificate may be used.

One challenge was understanding the full lifecycle rather than focusing only on certificate creation. It was necessary to consider issuance, validity dates, serial numbers, fingerprints, revocation, renewal, and evidence collection as one connected process. Another challenge was recognizing the difference between a demonstration and a production security service. The local prototype stores synthetic private keys and a simple revocation record so that the experiment is reproducible; a real CA would require protected key storage, role separation, signed status services, identity proofing, logging, monitoring, and incident-response procedures. Testing the same certificate before and after revocation was especially useful because it made the security effect visible and measurable.

The assignment improved my practical understanding of RSA key pairs, SHA-256 signatures, X.509 fields, trust anchors, certificate chains, and revocation decisions. It also strengthened my ability to convert a security requirement into requirements, architecture, pseudocode, source code, test cases, and validation evidence. The most important learning was that PKI security depends on both mathematics and governance: strong algorithms are insufficient if certificates are issued carelessly, private keys are exposed, or revocation information is unavailable.

The work connects to SDG 9 by supporting secure and resilient digital infrastructure, SDG 16 by improving authenticated digital identity and accountability, and SDG 17 by enabling trusted communication between organizations. It contributed to CO3 through the application of hashing, digital signatures, and X.509 authentication; to CO4 through secure certificate practices for network services; and to CO5 through analysis of impersonation, man-in-the-middle risks, revocation, and trust architecture. Overall, the assignment gave me a clearer understanding of how certificate management supports confidentiality, integrity, authentication, and non-repudiation in real networked systems.

15. References

[1]: <https://datatracker.ietf.org/doc/html/rfc5280> "RFC 5280: Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile"

[2]: <https://www.rfc-editor.org/rfc/rfc8446> "RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3"

[3]: https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html "OWASP Transport Layer Security Cheat Sheet"

[4]: <https://cryptography.io/en/latest/x509/> "Python Cryptography Documentation: X.509 Reference"

[5]: https://csrc.nist.gov/glossary/term/public_key_infrastructure_x_509_certificate_management_protocol "NIST Glossary: Public Key Infrastructure X.509 Certificate Management Protocol"

[6]: <https://sdgs.un.org/goals> "United Nations Sustainable Development Goals"

Appendix A – Run Instructions

```
cd pki_assignment
python3 pki_demo.py
python3 make_chart.py
```

```
#!/usr/bin/env python3
"""Miniature X.509 certificate management system for CSA51 Assignment 7.

The program creates a root CA, issues a server certificate, validates it,
revokes it, confirms that validation then fails, renews it, and measures the
main operations. It is intended for a controlled educational demonstration,
not production PKI.
"""
from __future__ import annotations

import csv
import hashlib
import json
import os
import shutil
import time
from datetime import datetime, timedelta, timezone
from pathlib import Path

from cryptography import x509
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.x509 import DNSName, NameOID
from cryptography.x509.oid import ExtendedKeyUsageOID

BASE = Path(__file__).resolve().parent
OUT = BASE / "artifacts"
KEYS = OUT / "keys"
CERTS = OUT / "certs"
for p in (KEYS, CERTS):
    p.mkdir(parents=True, exist_ok=True)

NOW = datetime.now(timezone.utc)

def save_key(key, path: Path):
```